

Java Language Evolution Dojo (with Exercises)

1. Functional Composition

Compose operations using streams and lambdas.

Exercise 1: Refactor imperative loop into stream pipeline

Exercise 2: Chain multiple transformations using map/flatMap

Exercise 3: Implement a reusable function composition utility

2. Bridge to Reactive Thinking

Understand pull vs push data flow.

Exercise 1: Simulate pull-based iteration vs push-based callback

Exercise 2: Convert a synchronous pipeline into async callbacks

Exercise 3: Identify bottlenecks in pull-based processing

3. Reactive Thinking

Build non-blocking pipelines.

Exercise 1: Design a simple event-driven system

Exercise 2: Implement retry logic in async flow

Exercise 3: Handle failures in a pipeline

4. Flow API

Use Publisher/Subscriber model.

Exercise 1: Implement a basic Publisher

Exercise 2: Add Subscriber with backpressure

Exercise 3: Create Processor transforming data

5. Enhanced Streams

Use takeWhile, dropWhile.

Exercise 1: Filter list using takeWhile

Exercise 2: Split stream using dropWhile

Exercise 3: Rewrite loops using enhanced streams

6. Optional Improvements

Leverage Optional effectively.

Exercise 1: Refactor null checks to Optional

Exercise 2: Chain Optional operations

Exercise 3: Use Optional.stream in pipelines

7. HTTP Client

Async HTTP requests.

Exercise 1: Call API asynchronously

Exercise 2: Aggregate multiple API responses

Exercise 3: Add timeout handling

8. Records

Use immutable data carriers.

Exercise 1: Convert POJO to record

Exercise 2: Add validation in record

Exercise 3: Compare record vs class usage

9. Sealed Classes

Control inheritance.

Exercise 1: Define sealed hierarchy

Exercise 2: Restrict subclasses

Exercise 3: Refactor inheritance to sealed model

10. Pattern Matching

Simplify instanceof.

Exercise 1: Replace instanceof casting

Exercise 2: Refactor polymorphic logic

Exercise 3: Compare with visitor pattern

11. Switch Expressions

Modern control flow.

Exercise 1: Rewrite switch statement

Exercise 2: Return values from switch

Exercise 3: Use pattern matching in switch

12. Virtual Threads

Lightweight concurrency.

Exercise 1: Run tasks using virtual threads

Exercise 2: Compare with thread pool

Exercise 3: Measure performance differences

13. Structured Concurrency

Group concurrent tasks.

Exercise 1: Run parallel tasks with structure

Exercise 2: Handle errors collectively

Exercise 3: Cancel group of tasks